

63. Les trois façons de faire la même chose

À la suite, par une fonction, par une méthode — et ce que chacune coûte.

1. Ce qu'il faut comprendre

À la suite, tout est dans `main` : `if (ennemiVie > 0) ennemiX++;`, avec deux variables globales. C'est le plus court, et pour **un** ennemi c'est le bon choix. Le jour où il en faut trois, il faut six variables et trois fois les mêmes lignes.

Par des fonctions : les données sont rangées dans un tableau de `struct`, le geste est nommé, et l'objet se désigne par son index — `avancer(i)`, avec `void avancer(uint8_t i) { if (troupe[i].vie > 0) troupe[i].x++; }`. Coût : un octet par argument.

Par des méthodes, ci-dessous : rigoureusement le même travail, mais le geste est écrit **là où vivent les données**, et l'objet ne se passe plus. Coût : deux octets par méthode.

Mesuré, sur ce programme : **1344 octets** à la suite, **1590** par fonctions, **1610** par méthodes. Aucune n'est meilleure dans l'absolu — la troisième rend en lisibilité ce qu'elle prend en octets, et c'est bien pour cela que le compilateur les compte et les annonce.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1643 octets** de cartouche.

1. UN ENREGISTREMENT — ENNEMI

```
struct Ennemi {
    uint8_t x, vie;

    void avancer() { if (vie > 0) x++; }
    void blesser() { if (vie > 0) vie--; }
};
```

2. UN TABLEAU — TROUPE

```
Ennemi troupe[3];
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

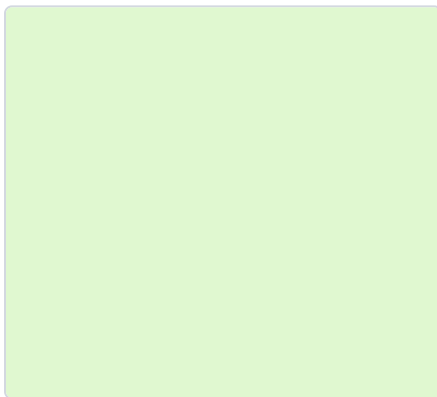
```
int main() {
    for (uint8_t i = 0; i < 3; i++) { troupe[i].x = 20 + i * 40; troupe[i].vie = 3; }
    uint8_t avant = 0;

    while (true) {
        uint8_t appui = bouton(A);
        for (uint8_t i = 0; i < 3; i++) {
            troupe[i].avancer();
            if (appui && !avant) troupe[i].blesser();
        }
        avant = appui;

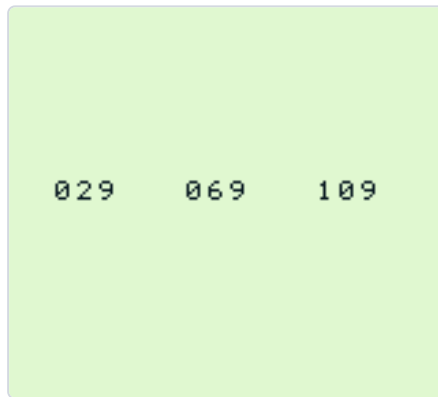
        for (uint8_t i = 0; i < 3; i++) nombre(2 + i * 6, 8, troupe[i].x);
        image();
    }
}
```

3. Ce qui se passe, étape par étape

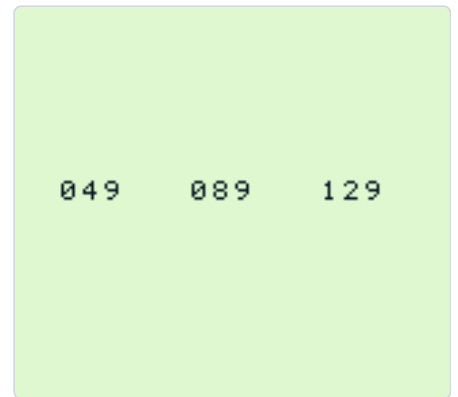
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



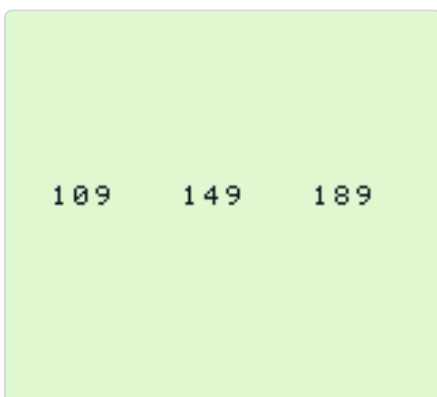
la toute première image — la console vient de démarrer — image 1



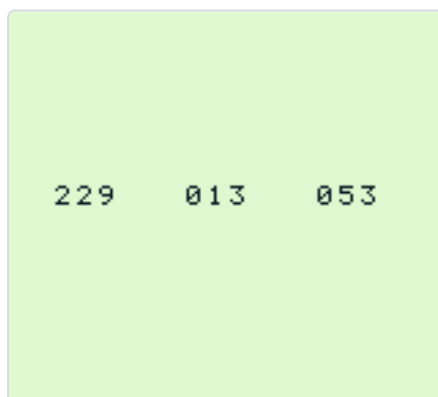
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



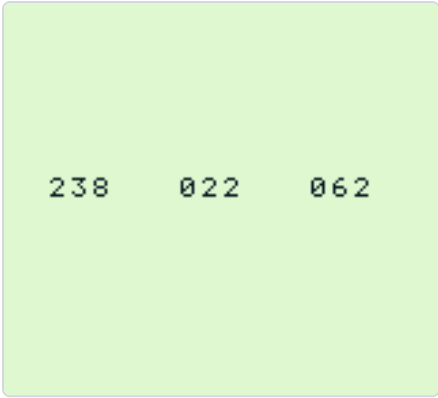
au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



238 022 062

A — l'écran a changé

5. Ce qu'on doit voir

Trois nombres qui montent ; A les blesse tous les trois.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les trois avancent (029 → 044)
- ✓ trois appuis les tuent
- ✓ et ils s'arrêtent alors
- ✓ les trois se comportent pareil

Ensuite : **64. Une méthode qui en appelle une autre** — niveau 8 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++